MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Sanjeev Khatri

Subject & Subject Code: **Semester:** OP using C++ - 3CS4-04 **Session:** 2023-24(Odd Sem.)

Unit No.: 2 **Date of Submission:**

Lect.No.	Topics Covered	Page No.	Total pages
1	Polymorphism : Binding, Static Binding, Dynamic Binding	1-5	5
2	Static Polymorphism	6-8	3
3	Function Overloading	9-12	4
4	Ambiguity in Function Overloading	13-14	2
5	Dynamic Polymorphism : Base Class Pointer, Late Binding	15-16	2
6	Object Slicing	17-22	6
7	Method Overriding with Virtual Functions	23-27	5
8	Pure Virtual Functions	28-31	4
9	Abstract Classes	32-35	4

References Book:

1. Object Oriented Programming with C++ - E Balagurusamy
2. C++ : The Complete Reference – Herbert Schlitz
3. Object Oriented Programming in Turbo C++ - Robert Lafore.
4. C++ How to Program – Paul Dietel

05/10/2023

FACULTY **HOD** **DEAN ACADEMICS**

UNIT-2

LECTURE - 1

Polymorphism

Polymorphism

Polymorphism word is the combination of "poly" which means many + "morphs" which means forms, which together means many forms. Polymorphism in C++ is when the behavior of the same object or function is different in different contexts.

It is of two types – 1. Compile Time Polymorphism 2. Runtime Polymorphism

1. Compile-time polymorphism is done by overloading an operator or function. It is also known as **static** or **early binding**.
2. Runtime polymorphism occurs when functions are resolved at runtime rather than compile time, when a call to an overridden method is resolved dynamically at runtime rather than compile time. It's also known as **late binding** or **dynamic binding**.

Static or Early Binding-

Static polymorphism is additionally termed as compile-time polymorphism, which implies that one can write numerous methods in a program with the same name, performing distinctive tasks. However, it gives the client or the software engineer efficient and better comprehensibility of code.

Example-

```
#include <iostream>

using namespace std;

class Calcs {

public:

    void calc(int a) //Function with Single parameter - int type

    {

        cout << "Value of a is " << a << endl;

    }


    void calc(double a) //Function with Single parameter - double type
```


MITRC

```
        {
                cout << "Value of a is " << a << endl;
        }

        void calc(int a, int b) //Function with more than one parameter - int, int
        {
                cout << "Value of a and b is " << a << ", " << b << endl;
        }
};

//Main Function
int main()
{
        Calcs calcobj;

        calcobj.calc(5);
        calcobj.calc(3.121);
        calcobj.calc(77, 34);
        return 0;
}
```

Dynamic or Late Binding-

Dynamic binding means the code associated with a given procedure call is not known untill the time of the call at run-time. It is associated with a polymorphic reference depends upon the dynamic type of that reference. It is a procedure in which a call to an overridden method is settled at runtime, which is the reason it is termed as runtime polymorphism.

Example-

```
#include <iostream>
```

NITRC

```
#include <conio.h>
using namespace std;
class baseA
{
   public :
      virtual void display ()
      {
         cout<< "Base Class-A \n";
      }
};

class derivedB : public baseA
{
   public:
      virtual void display()
      {
         cout<< "Derived Class-B \n";
      }
};

class derivedC: public derivedB
{
   public:
      virtual void display ( )
      {
         cout<< "Derived Class-C \n";
      }
```

MITRC

```
};


void main ( )
{
   //define three objects
   baseA obja;
   derivedB objb;
   derivedC objc;
   baseA *ptr[3]; //define an array of pointers to baseA
   ptr [0] = &obja;
   ptr [1] = &objb;
   ptr [2] = &objc;
   for (int i = 0; i <=2; i ++)
      ptr[i]->display(); //same message for all objects


   obja.display();
   objb.display();
   objc.display();


   getche();
}
```

Key Differences-

Static or Early Binding	Dynamic , Run Time or Late Binding
The call is settled by the compiler.	The call isn't settled by the compiler.
It is otherwise called as Compile-time Polymorphism and Early binding	it is otherwise called Dynamic binding, Late binding and overriding also
Overloading is compile-time polymorphism where more than one functions share a	Overriding is run time polymorphism having the same function with the same

MITRC

similar name with various parameters or signature and distinctive return type,	parameters or mark, however, related in a class and its subclass.
Static polymorphism is accomplished by function overloading and operator overloading	Runtime polymorphism is accomplished by pointers and virtual functions.
As in static polymorphism, it is analyzed early at compile time so it provides fast execution.	Runtime polymorphism is slow because it is analyzed at the runtime.

MITRC

LECTURE - 2

Static Polymorphism

Static Polymorphism is the linking of a function with an object during compile time is called static. Static Polymorphism can check assumptions at compile time.

In simple words we can say that an object is bound to its function call at the compile time.

Example-1

```
#include<iostream>
using namespace std;

class B
{
   public:
      void show()
      {
         cout << "Base Class \n";
      }
};

class D:public B
{
   public:
      void show()
      {
         cout << "Derived Class \n";
      }
};
```

MITRC

```
void main()
{
   D d;
   d.show(); //Calls Derived class show() function
   d.B::show(); //Calls Base class show() function
}
```

Example-2

```
#include<iostream>

using namespace std;

class Calc
{
   private:
    void show(int d)
   {
      cout<<"Sum of two values : "<<d<<"\n";
   }
   public:
      void sum(int a, int b)
      {
         int c = a+b;
         show(c);
      }
```

MITRC

```
    int sum(int a, int b, int c)
    {
       return a+b+c;
    }

};

void main()
{
   Calc clc;
   clc.sum(5,10);
   int d = clc.sum(5,10,2);
   cout << "Sum of Three Values : "<<d<<"\n";
}
```

Ways to Implement Compile-time Polymorphism

Function Overloading

Function Overloading takes place when you create two functions of the same name and these functions serve different purposes. Both functions should have the same return types and strictly different arguments from each other.

Operator Overloading

In operator overloading, different operators display different implementations based on their parameters or signatures. The C++ programming language allows you to create operators that can serve specific purposes with user-defined functions.

MITRC

LECTURE - 3

Function Overloading

Function Overloading takes place when you create two or more functions of the same name and these functions serve different purposes. Both functions should have the same return types and strictly different arguments from each other.

Readability of the program increases by function overloading. It is achieved by using the same name for the same action.

```
// Function with the same name and different arguments.
int area(int a) {
  // Function with one integer parameter.
}
int area(int a, int b) {
  // Function with two integer parameters.
}
double area(double a) {
  // Function with one parameter of type double.
}
double area(float a, float b) {
  // Function with two parameters of type double.
}
```

Example-1

```
#include <iostream>
using namespace std;
class bird //Base Class
{
  public:
```

MITRC

```
   void show() //Show Function of the Base Class
   {
      cout << "I am the show function of the base class - Bird \n";
   }
};

class parrot:public bird //Derived Class
{
  public:
   void show() //Show Function of the Derived Class
   {
      cout << "I am the show function of the derived class - Parrot\n";
   }
};

int main()
{
  bird b;
  parrot p;
  b.show();
  p.show();
  return 0;
}
```

Example-2

```
#include<iostream>
using namespace std;
```

MITRC

```
void repchar(); //Without Parameters
void repchar(char); //With One Parameter
void repchar(char, int); //With Two Parameters

int main()
{
   repchar();
   repchar('=');
   repchar('+', 30);
   return 0;
}
// displays 45 asterisks
void repchar()
{
   for(int j=0; j<45; j++) // always loops 45 times
      cout << '*'; // always prints asterisk
   cout << endl;
}
// displays 45 copies of specified character
void repchar(char ch)
{
   for(int j=0; j<45; j++) // always loops 45 times
      cout << ch; // prints specified character
   cout << endl;
}
// displays specified number of copies of specified character
void repchar(char ch, int n)
```

MITRC

```
{
   for(int j=0; j<n; j++) // loops n times
      cout << ch; // prints specified character
   cout << endl;
}
```

Advantages of Function Overloading-

- ❖ It improves code readability.
- ❖ It speeds up the execution of the program.
- ❖ It allows code reusability.
- ❖ It is used to save memory space, consistency, and readability.
- ❖ Overloading is done during compile time, and the compiler knows which function is getting called before execution.

MITRC

LECTURE - 4

Ambiguity in Function Overloading

When many functions have the same name but different signatures or parameters, this is known as Function Overloading. The C++ Function Overloading feature is used to make the code easier to read.

Overloading of functions must have:

- More than one function definition uses the same function name.
- Different parameters must be used with overloaded functions.

In C++, there are two kinds of Function Overloading.

Overloading at compile time: Different signatures are used to overload functions during compile time overloading. A function's return type, parameter count, and parameter type are all part of its signature.

Runtime Overloading - Runtime Overloading involves using a varying amount of parameters to overload functions.

Function overloading and Ambiguity

In Function overloading, sometimes a situation can occur when the compiler is unable to choose between two correctly overloaded functions. This situation is said to be ambiguous. Automatic type conversions are the main cause of ambiguity.

Example-

```
#include <iostream>

using namespace std;

class demo

{

  public:

    int main_demo(int a)

    {

       Cout << "a = " << a << endl;
```

MITRC

```
        return 0;
    }
    int main_demo(int a, int b)
    {
        cout<<"a = "<<a<<"; b = "<<b<<endl;
        return 0;
    }
};
int main() {
   demo object;
   object.main_demo(5);
   object.main_demo(5,10);
   object.main_demo('a');
   return 0;
}
```

In the example above, object.main_demo('a') is an ambiguity. Because, we need to define an int type data for main_demo() function call but we are suppling a char type data. In this case, compiler will not generate any error and provide us output for it.

Output-

a=5

a=5; b=10

a=97 → this a char value of a instead of actual output.

Or some compilers may generate error like-

error: call of overloaded main_demo (char) is ambiguous.

Why this happens-

When there is no exact type match, the compiler looks for the closest match.

NITRC

LECTURE - 5

Dynamic Polymorphism : Base Class Pointer

It is also known as Runtime Polymorphism. In this case, the funciton call is resolved at run time. It is achieved by virtual functions and pointers.

Late Binding

In this type of binding the compiler adds code that identifies the object type at runtime then matches the call with the right function definition. This is achieved by using virtual function.

Virtual Function

Virtual functions, is the one of advanced features of OOP that does not really exist but it appears real in some parts of a program. This section deals with the polymorphism features which are incorporated using the virtual functions.

Pointers

A pointer stores the memory address as its value. The address of a variable can be assigned to a pointer variable with the help of & operatror.

The general syntax of Pointer declaration is:

datatype *pointer_variable;

or

datatype* pointer_variable;

Example-

int normal_variable = 7;

int* pointer_variable;

pointer_variable = &normal_variable;

An Example showing use of Dynamic Polymorphism-

#include <iostream>

using namespace std;

MITRC

```
class Base1
{
   public:
      void show()
      {
         cout << "Base-1 Class function"<<endl;
      }
};

class Child1:public Base1
{
   public:
      void show()
      {
         cout << "Child-1 Class function"<<endl;
      }
};

int main()
{
   Child1 ch1;
   Base1 bs1;
   Base1* bs1ptr =  &ch1; //Taken Memory Address of Child-1 Class to call Base Class Function
   bs1ptr->show(); //Call of Base Class function through Child-1 Class
   ch1.show();
   return 0 ;
}
```

MITRC

LECTURE - 6

Object Slicing

When a derived class object is assigned to a base class object, the additional attributes of a derived class object are sliced off (not considered) to form the base class object, is called Object Slicing. When pointers or references to objects are passed as function arguments since a pointer or reference of any type takes same amount of memory, Object slicing does not occur.

Example-

```
#include <iostream>

using namespace std;


class Base

{

   protected:

      int l;


   public:

      Base(int m)

      {

         l = m;

      }

      virtual void display()

      {

         cout << "This is Base Class Object  l = " << l << endl;

      }

};


class Derived : public Base
```

MITRC

```
{
   int n;

   public:
      Derived(int l, int m): Base(l)
      {
         n = m;
      }
          virtual void display()
          {
                    cout << "This is Derived Class Object l = " << l  << ", n = " << n << endl;
          }
};

void somefunc(Base bsobj)
{
   bsobj.display();
}

int main()
{
          Base bs(1);
          Derived dr(2, 3);
          somefunc(bs);
          somefunc(dr); // Object Slicing - the member n of dr is sliced off
          return 0;
}
```

MITRC

Output-

This is Base Class Object l =1

This is Base Class Object l = 1

In this example, the member n of derived class is sliced off.

To avoid this situation, we can use reference, like-

```
#include <iostream>
using namespace std;

class Base
{
   protected:
      int l;

   public:
      Base(int m)
      {
         l = m;
      }
      virtual void display()
      {
         cout << "This is Base Class Object l = " << l << endl;
      }
};

class Derived : public Base
```

MITRC

```
{
        int n;

   public:
      Derived(int l, int m): Base(l)
      {
         n = m;
      }
         virtual void display()
         {
                cout << "This is Derived Class Object l = " << l << ", n = " << n << endl;
         }
};

void somefunc(Base &bsobj)
{
   bsobj.display();
}

int main()
{
         Base bs(1);
         Derived dr(2, 3);
         somefunc(bs);
         somefunc(dr);
         return 0;
}
```

Output-

MITRC

This is Base Class Object l = 1

This is Derived Class Object l = 2, n = 3

Or, we can use pointers, like-

```
#include <iostream>
using namespace std;

class Base
{
   protected:
      int l;

   public:
      Base(int m)
      {
         l = m;
      }
      virtual void display()
      {
         cout << "This is Base Class Object l = " << l << endl;
      }
};

class Derived : public Base
{
         int n;
   public:
      Derived(int l, int m): Base(l)
```

MITRC

```
    {
       n = m;
    }
        virtual void display()
        {
                cout << "This is Derived Class Object l = " << l << ", n = " << n << endl;
        }
};

void somefunc(Base *bsobj)
{
   bsobj->display();
}

int main()
{
   Base *bs = new Base(1) ;
   Derived *dr = new Derived(2, 3);
        somefunc(bs);
        somefunc(dr); // Object Slicing - the member n of dr is sliced off
        return 0;
}
```

Output-

This is Base Class Object l = 1

This is Derived Class Object l = 2, n = 3

MITRC

LECTURE - 7

Method Overriding with Virtual Functions

Virtual Function-

Virtual functions, is the one of advanced features of OOP that does not really exist but it appears real in some parts of a program. The keyword virtual precedes the return type of the function name. It tells the compiler to perform late binding where the compiler matches the object with the right called function and executes it during the runtime.

The general syntax of the virtual function declaration is:

```
class class_name
{
 private:
 public:
  virtual return_type function_name1 (arguments);
  virtual return_type function_name2(arguments);
  virtual return_type function_name3( arguments);
};
```

To make a member function virtual, the keyword virtual is used in the methods while it is declared in the class definition but not in the member function definition.

For **example**-

```
class calc
{
        int x;
        int y;
        public:
```

MITRC

```
        virtual int z();
        virtual void show();
};
```

Example-

```
#include <iostream>
using namespace std;

class Base1
{
   public:
      void show()
      {
         cout << "Base-1 Class function"<<endl;
      }
};

class Base2
{
   public:
      virtual void show()
      {
         cout << "Base-2 Class function"<<endl;
      }
};

class Child1:public Base1
```

MITRC

```
{
   public:
      void show()
      {
         cout << "Child-1 Class function"<<endl;
      }
};

class Child2:public Base2
{
   public:
      void show()
      {
         cout << "Child-2 Class function"<<endl;
      }
};

int main()
{
   Child1 ch1;
   Child2 ch2;
   Base1 bs1 = ch1;
   Base2 bs2 = ch2;
   Base1* bs1ptr =  &ch1; //Taken Memory Address of Child-1 Class to call Base Class Function
   Base2* bs2ptr =  &ch2; //Taken Memory Address of Child-2 Class to call Base Class Function
   bs1ptr->show(); //Call of Base Class function through Child-1 Class
   bs2ptr->show(); //Call of Base Class function through Child-2 Class using virtual function
```

MITRC

```
    ch1.show();
    ch2.show();
    return 0 ;
}
```

A virtual function cannot be a static member since a virtual member is always a member of a particular object in a class rather than a member of the class as a whole.

```
class calc
{
        int a;
        int b;
        public:
                virtual static int z(); //error
};
int x:z()
{
        Function body;
}
```

A virtual function cannot have a constructor member function but it can have the destructor member function.

```
class calc
{
        int a ;
        int b ;
        public:
                virtual calc (int aa, int bb) ; // error using constructor
                void show ( ) ;
                int z ( ) ;
```

MITRC

```
};
```

A destructor member function does not take any argument and no return type can be specified for it not even void.

```
class calc
{
        int a ;
        int b ;
        public:
                virtual calc (int aa, int bb) ;  //wrong
                void show ( ) ;
                int z ( ) ;
};
```

It would be an error to redefine a virtual method with a change of return data type in the derived class with the same parameter types as those of a virtuall method in the base class.

```
class base
{
        int a, b ;
        public:
                virtual int add (int aa, int bb ) ; //error
} ;
class child: public base
{
        Int c ;
        public:
                virtual float add (int aa, int bb) ;
};
```

In the above example, use of two virtual functions is invalid even though these functions have identical arguments and different return data types.

MITRC

LECTURE - 8

Pure Virtual Functions

Virtual functions, is the one of advanced features of OOP that does not really exist but it appears real in some parts of a program. The keyword virtual precedes the return type of the function name.

Generally a function is declared virtual inside a base class and we redefine it in the derived class. A pure virtual function is a virtual function that has no definition within the class.

Characteristics of a Pure Virtual Function

- It just provides the template, and derived class implements the function.
- It does not have any definition relative to the base class.
- If any class containing any pure virtual function, then we cannot create the object of that class. This type of class is known as an abstract class.

Syntex-

```
virtual return_type function_name() = 0;
```

or

```
virtual return_type function_name() {};
```

Example-

```
#include<iostream>
#include<conio.h>

using namespace std;

class employee  //Base Class
{
   int code;
```


MITRC

```
    public:
        virtual void getempdtl() ;
        virtual void show() ;
};

class grade:public employee //Derived Class
{
    char name[50] ;
    char grad[50];
    float sal;
    public :
        void getempdtl() ;
        void show();
};

//Function getempdtl declared in employee Class without function body
void employee::getempdtl()
{

}

//Function show declared in employee Class without function body
void employee::show()
{

}
```

NITRC

```
//Function getempdtl declared in grade Class with function body
void grade::getempdtl()
{
   cout << "Enter Employee's Name : ";
   cin >> name ;
   cout << "\nEnter Employee's Grade : ";
   cin >> grad ;
   cout << "\nEnter Salary : " ;
   cin >> sal;
}
//Function show declared in grade Class with function body
void grade::show()
{
   cout << "Employee's Name   : " << name << endl;
   cout << "Employee's Grade  : " << grad << endl;
   cout << "Employee's Salary : " << sal << endl;
}

void main()
{
   employee *pntr; //Pointer for employee class
   grade obj ;
   pntr = &obj ;    //Derived class assigned to base class pointer
   pntr->getempdtl();
   pntr->show();
   getche() ;
}
```

MITRC

In above example, there are two classes employee and grade. Employee class is base class and grade class is derived. Functions getempdtl() and show() are declared for both classes. In Employee class, we have defined functions with empty body. In derived class we have written the desired code. The methods of the derived class are invoked by the pointer to the base class.

Difference between Virtual Function and Pure Virtual Function

Virtual Function	Pure Virtual Function
A virtual function is a member function of a base class and it can be redefined in a derived class.	It is a member function of a base class which is declared in a base class and implemented in a derived class.
The classes containing virtual functions, are not abstract classes.	The classes containing pure virtual functions, are abstract classes.
Definition of a virtual function is provided in the base class.	Definition of a pure virtual function is not provided in the base class.
In case of virtual functions, All the derived classes may or may not redefine the virtual function.	In case of pure virtual functions, All the derived classes must define the pure virtual function.

MITRC

LECTURE - 9

Abstract Classes

A class which contains atleast one Pure Virtual function in it is known as Abstract Class. Because an abstract class contains one or more functions for which there is no definition (that is, a pure virtual function), no objects of an abstract class may be created. These are used to provide an Interface for its sub classes.

Classes inheriting an Abstract Class must provide definition to the pure virtual function, otherwise they will also become abstract class. Although we cannot create objects of an abstract class, we can create pointers and references to an abstract class and this allows abstract classes to support run-time polymorphism.

Characteristics of Abstract Class

1. Abstract class cannot be instantiated, but pointers and refrences of Abstract class type can be created.
2. Abstract class can have normal functions and variables along with a pure virtual function.
3. Abstract classes are mainly used for Upcasting, so that its derived classes can use its interface.
4. Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.
5. An important use of abstract classes and virtual functions is in class libraries. We can create a generic, extensible class library that will be used by other programmers.

Example with Error-

```
#include<iostream>

#include<conio.h>

using namespace std;


class Base //Abstract base class

{

   public:
```


MITRC

```
    virtual void show() = 0; //Pure Virtual Function
};

class Derived:public Base
{
  public:
    void show()
    {
      cout << "Implementation of Virtual Function in Derived class";
    }
};

int main()
{
  Base objs; //Run Time / Compile Time Error (Subject to Compiler using)
  Base *bs;
  Derived dr;
  bs = &dr;
  bs->show();
  return 0;
}
```

Example- Showing Abstract Class using Virtual function.

```
#include <iostream>
using namespace std;

class convert
```

MITRC

```
{
   protected:
      double val1; // initial value
      double val2; // converted value
   public:
      convert(double i)
      {
         val1 = i;
      }
      double getconv() { return val2; }
      double getinit() { return val1; }
      virtual void compute() = 0;
};

// Liters to gallons.
class l_to_g : public convert
{
   public:
   l_to_g(double i) : convert(i) { }
   void compute()
   {
      val2 = val1 / 3.7854;
   }
};

// Fahrenheit to Celsius
class f_to_c : public convert
```

MITRC

```
{
   public:
      f_to_c(double i) : convert(i) { }
      void compute()
      {
         val2 = (val1-32) / 1.8;
      }
};
int main()
{
   convert *p; // pointer to base class
   l_to_g lgob(4);
   f_to_c fcob(92.5);
   // use virtual function mechanism to convert
   p = &lgob;
   cout << p->getinit() << " liters is ";
   p->compute();
   cout << p->getconv() << " gallons\n"; // l_to_g
   p = &fcob;
   cout << p->getinit() << " in Fahrenheit is ";
   p->compute();
   cout << p->getconv() << " Celsius\n"; // f_to_c
   return 0;
}
```

In above example The base class convert is an example of an abstract class. The virtual function compute( ) is not defined within convert because no meaningful definition can be provided. The class convert simply does not contain sufficient information for compute( ) to be defined.

MITRC